



MiniMax-H3 on vLLM-Omni

Day-0 support and community optimization

vLLM-Omni Diffusion · August 2026

Agenda

- 01** **Minimax-H3 overview and Day-0 support on vLLM-Omni**

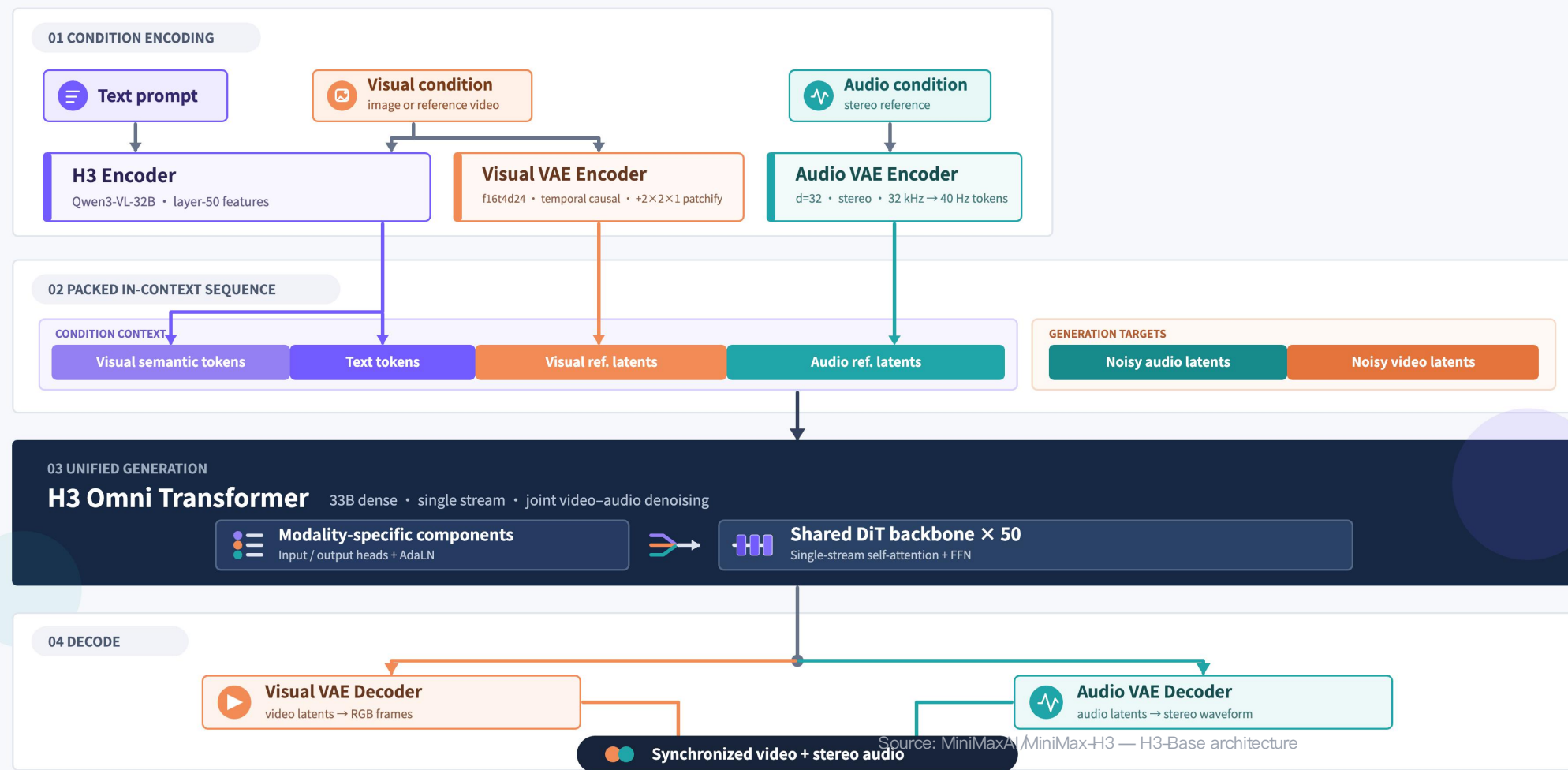
- 02** **Minimax-H3 Day-1 optimization on vLLM-Omni**

- 03** **Minimax-H3 on Consumer GPUs: The magic of Distributed Layerwise Offload**

Minimax-H3 overview and Day-0 support

Optimization after Minimax-H3 day0 support

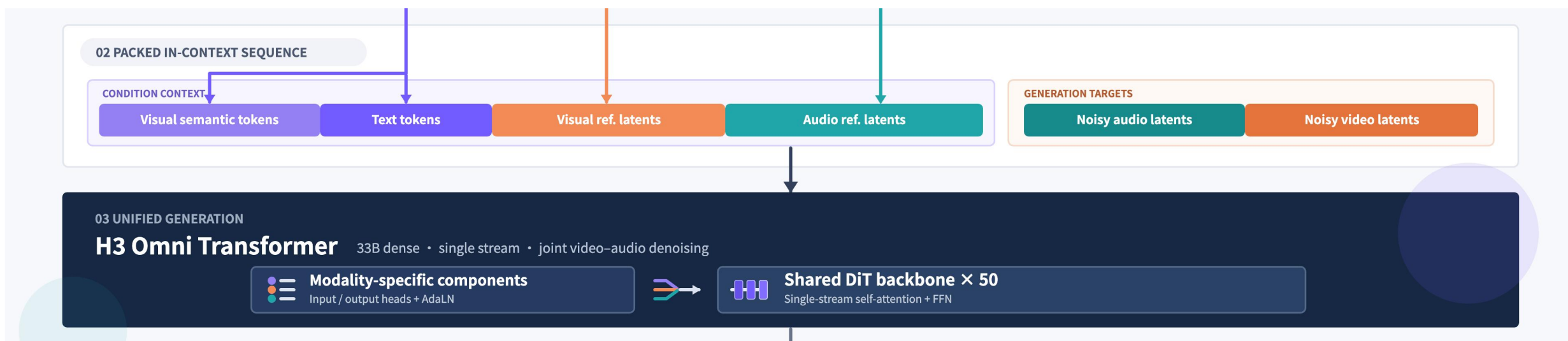
MiniMax H3-Base



Single stream DiT with packed seq

What stood between the release and a working service

- 32B Qwen3-VL text encoder + 33B DiT for FL2VA/Ref2VA respectively
- Long packed multimodal sequence inputs + single-stream self-attention

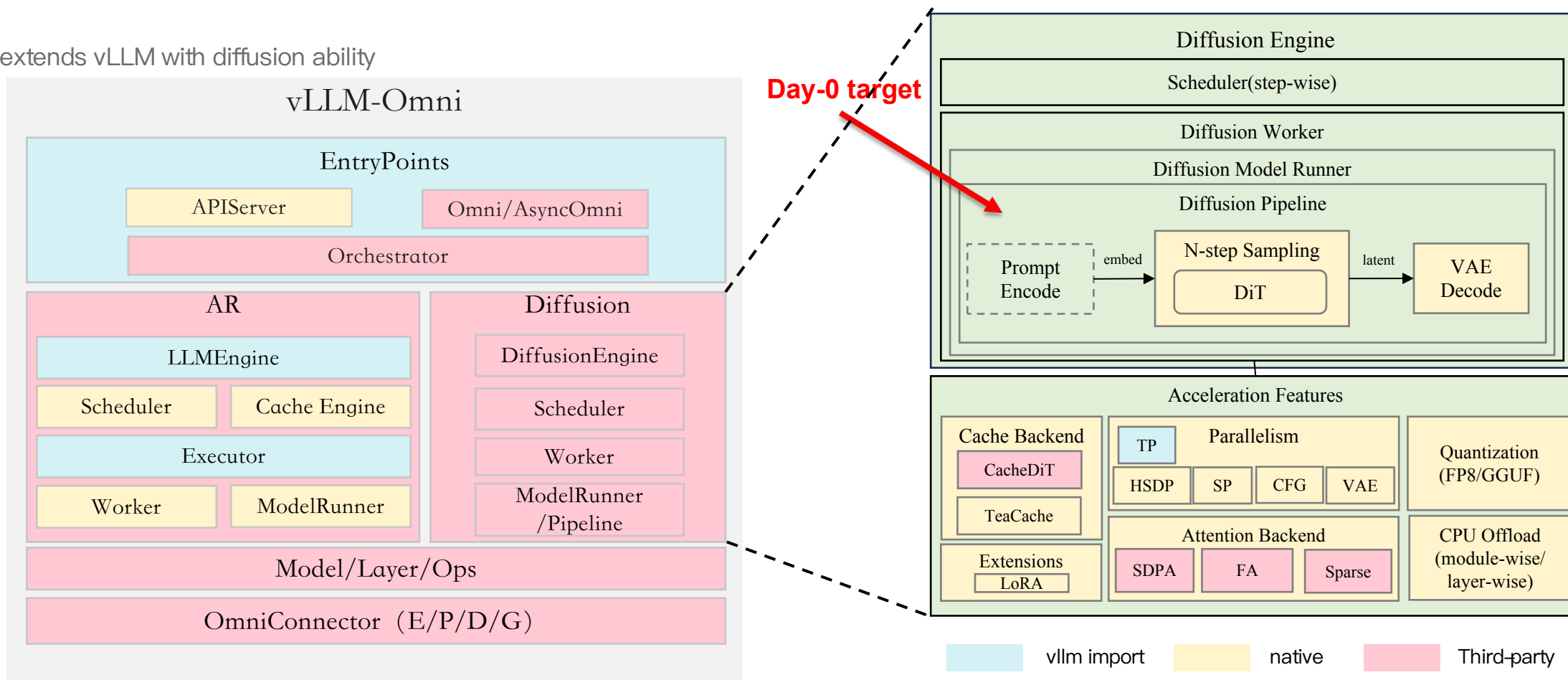


Source: MiniMaxAI/MiniMax-H3 — H3-Base architecture

DAY-0

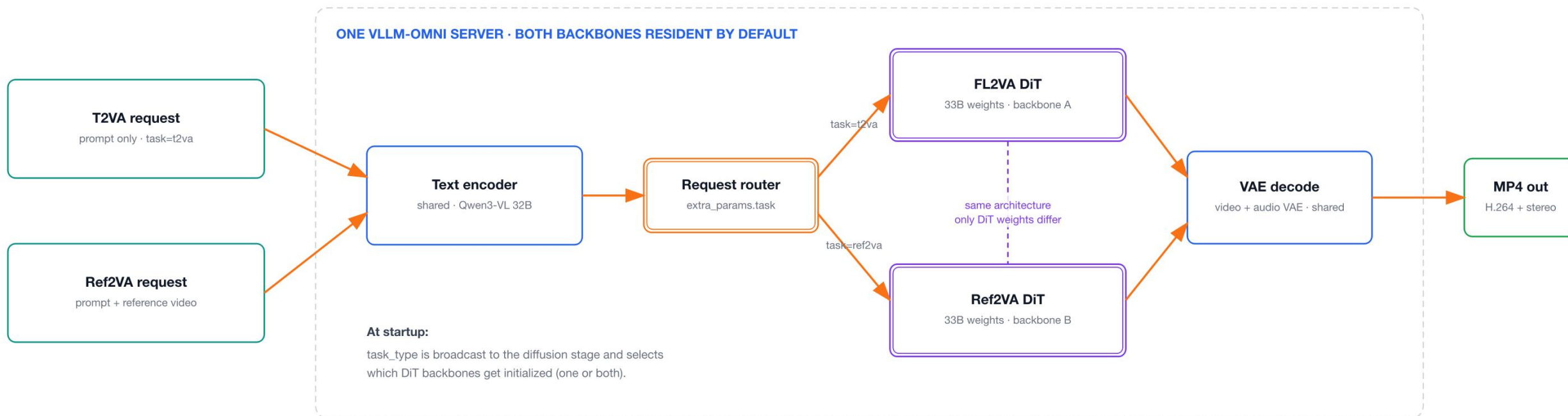
vLLM-Omni

vLLM-Omni extends vLLM with diffusion ability



Task-routed pipeline

Share text encoder /vae for FL2VA and Ref2VA



default: both DiTs resident, one endpoint serves every task · --task-type fl2va | ref2va loads a single DiT to save VRAM

Encoder TP + DiT SP

What stood between the release and a working service

- Encoder TP: Qwen3-VL encoder weights shards across N ranks (`--text-encoder-tp-size N`)
- Ulysses SP: Splits long sequence N ways; two All-to-All per attention



Encoder TP splits weights (capacity) · USP splits activations (sequence length) — they compose: 4xB300 recipe runs USP4 + encoder TP4 + VAE PP4

Varlen Attention Backend

What stood between the release and a working service

- Invariant batch attention no longer efficient

- Varlen attention backend:

Flash Attention / TRTLLM / CUDNN

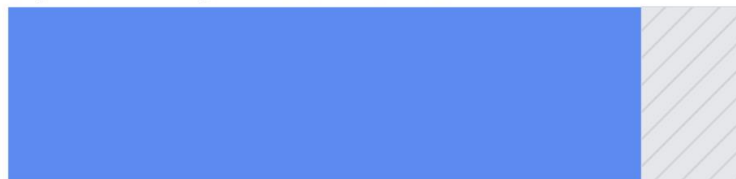
H3 packs all modalities into one sequence — and T is different for every request

Fixed-shape kernels: pad every request to T'
pay compute + memory for the hatched area

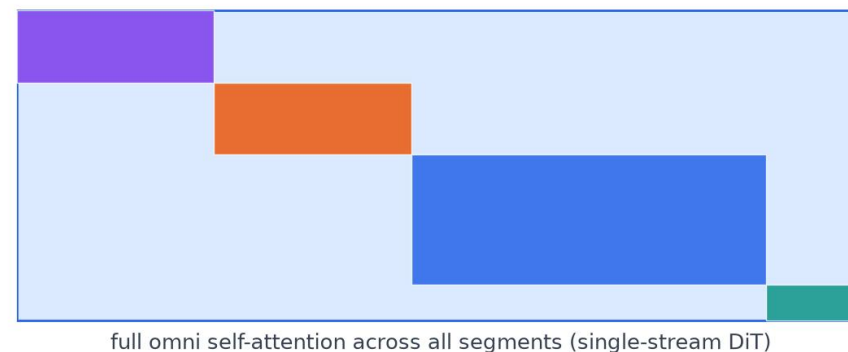
request A: T=38 padded to fixed T'=50



request B: T=43 padded to fixed T'=50



Varlen attention: exact T per request, no padding
(full joint attention — that is the omni fusion)



text video audio ref

Why choose TRTLLM_ATTN by default

Matched A/B on 4× B300 — identical recipe and seed; the attention backend is the only variable

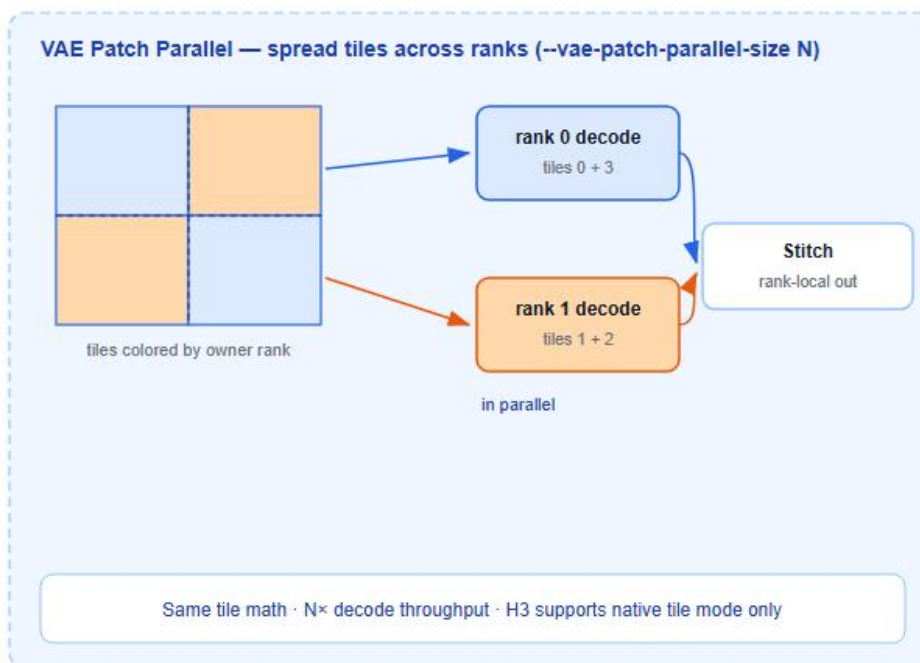
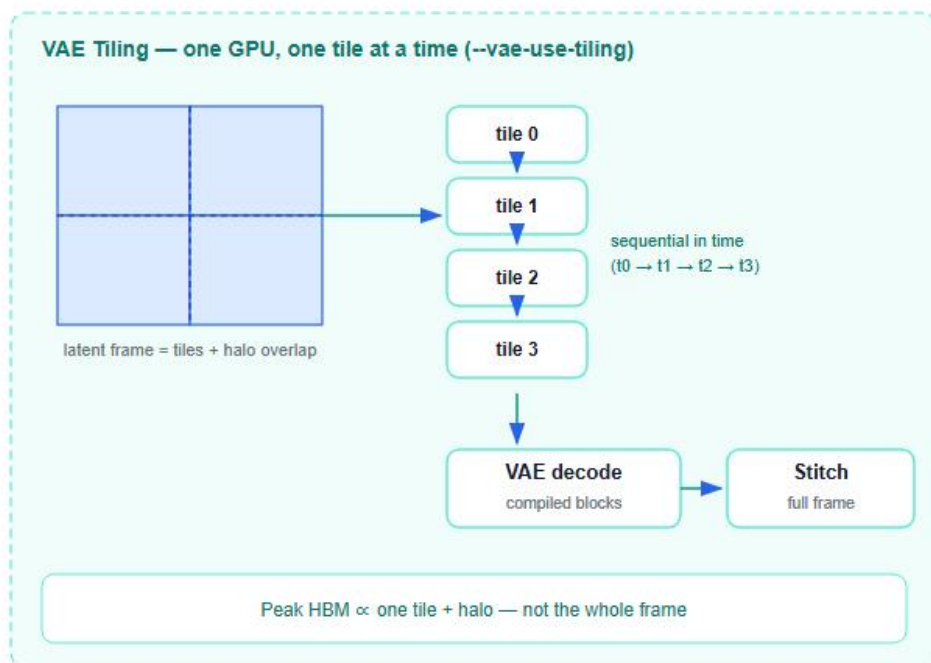
- Default: `--diffusion-attention-backend TRTLLM_ATTN`
- Taks: FL2VA T2VA, 1248×768, 209 frames + 32 kHz stereo, 50 steps, seed 1101
- Setup: TP1 · Ulysses4 · Ring1 · encTP1 · VAE tile4 + regional compile
- **Packed kernels: Skip-Softmax + Sage Attention**

Backend	Diffusion	End-to-end
FLASH_ATTN (FA4)	83.85 s	88.56 s
TRTLLM_ATTN (default)	71.99 s	76.18 s
TRTLLM vs FA4	-14.2%	-14.0%

VAE Tiling & Patch Parallelism

What stood between the release and a working service

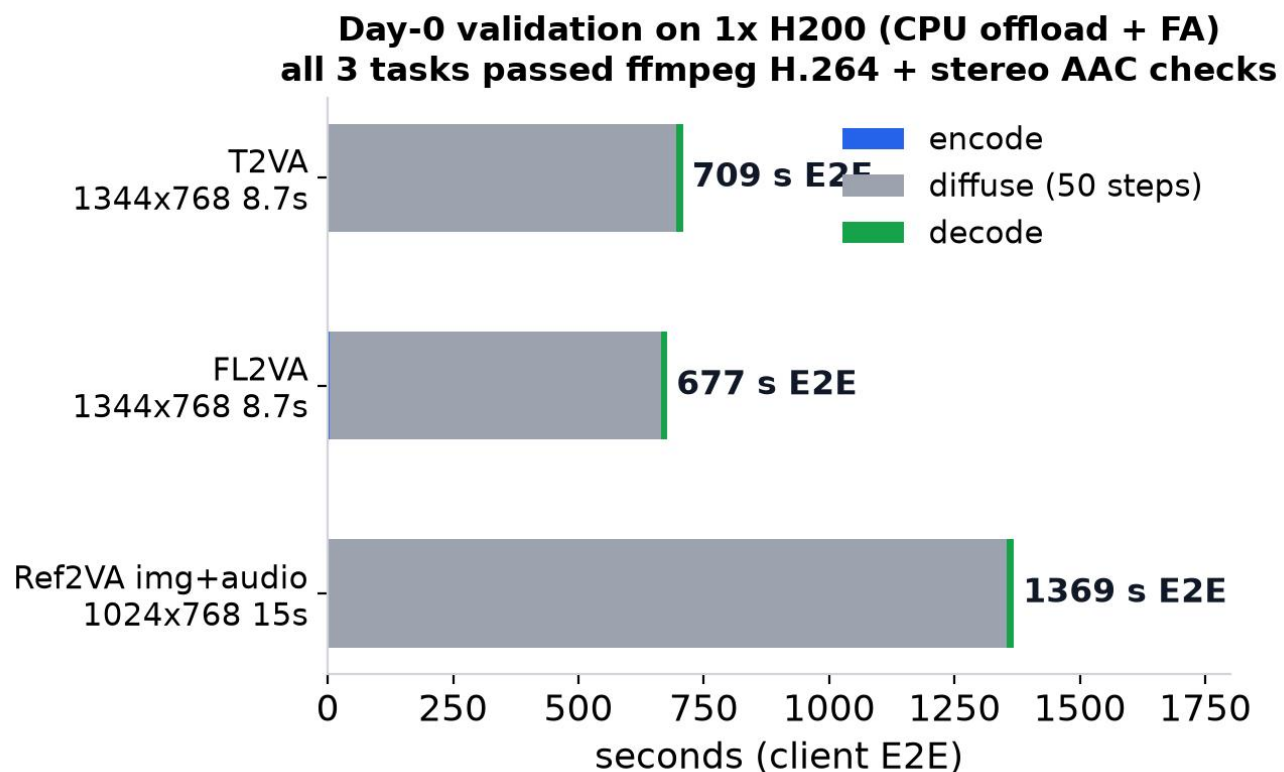
- Tiling (`--vae-use-tiling`): decode the latent frame tile by tile with halo overlap — peak HBM $\propto$ one tile
- Patch parallel (`--vae-patch-parallel-size N`, tile mode): tiles are dealt to N ranks and decoded in parallel



DAY-0

Day-0 Validation on 1 × H200

T2VA / FL2VA / Ref2VA all pass end-to-end with byte-level stream checks



Source: PR #5691 validation runs · single H200, CPU offload

Day-1 optimization

Optimization after Minimax-H3 day0 support

OPTIMIZATION

Kernels fusion

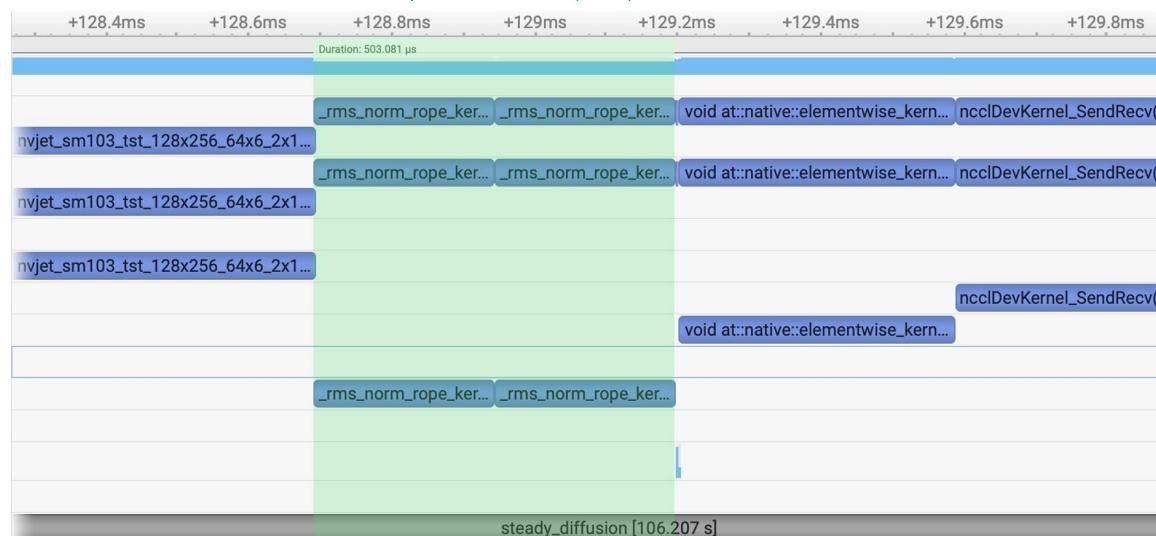
Faster BF16 attention first; sparse attention next

- Fused Q/K RMSNorm + packed RoPE (#5990, #5801)
- Kernel wall time: 109.7s $\rightarrow$ 106.2s (-3.2%)

BEFORE: separate triton kernels - denoise 109.7 s



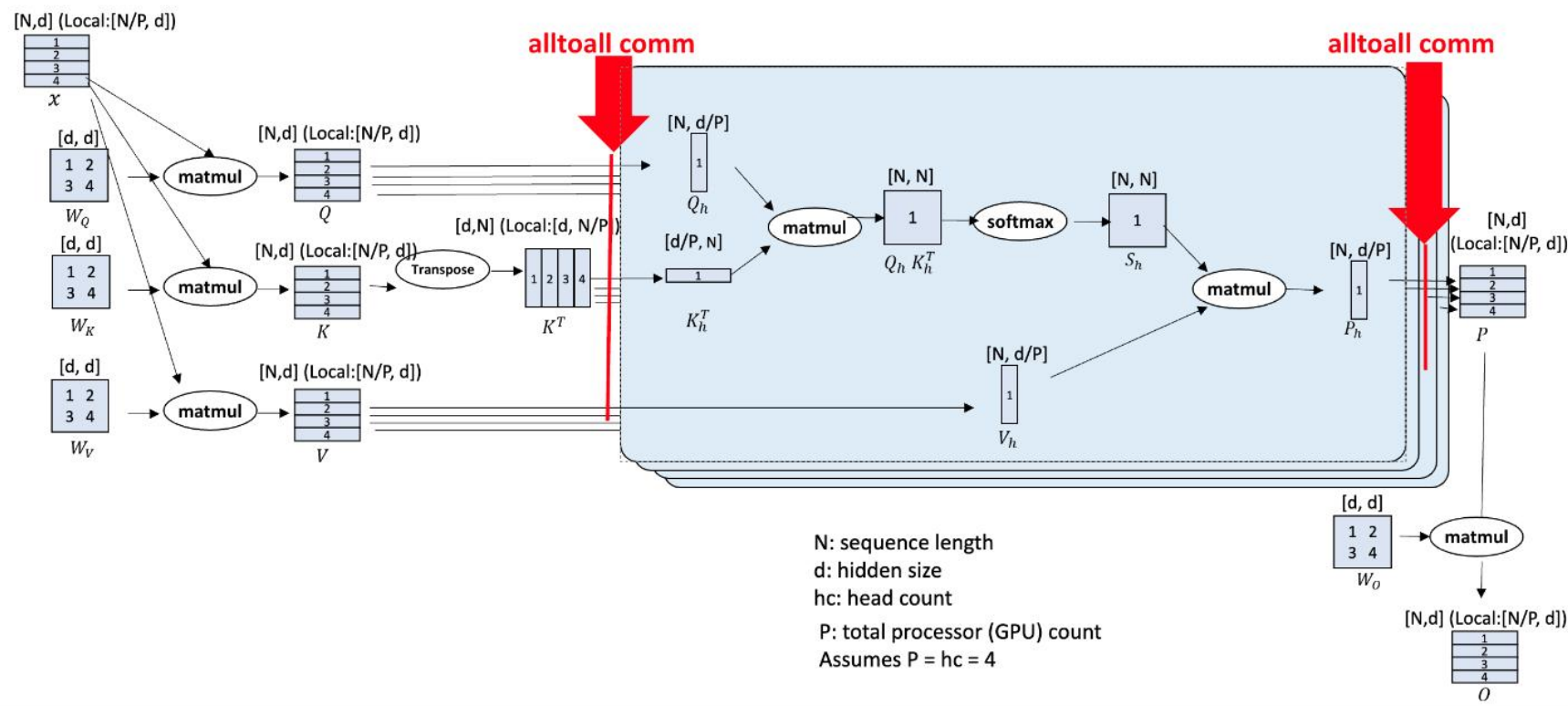
AFTER: fused Q/K RMSNorm + RoPE custom op - denoise 106.2 s (-3.2%)



Source: PR #5990 · profiler traces, 4×B300, 1344×768 × 243f × 50 steps

Strict Ulysses Boundaries

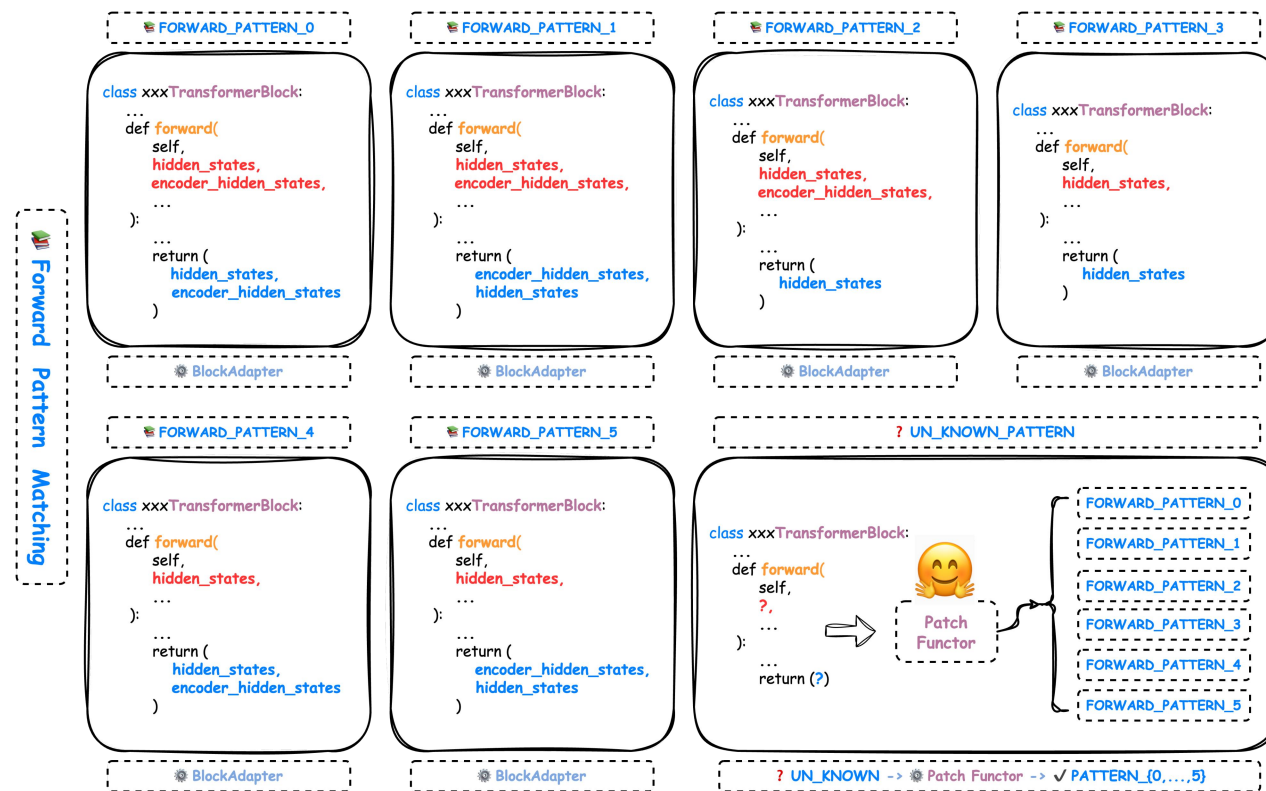
Project locally, then AllGather compact logits — not the full hidden states ($\#6173$) · $21 \times$ smaller payload, ~ 1.24 MiB/rank



Source: PR #6173 · figure: Ulysses sequence parallelism (DeepSpeed)

Forward-pattern matching decides which blocks can skip recompute

- quality = lossless | high



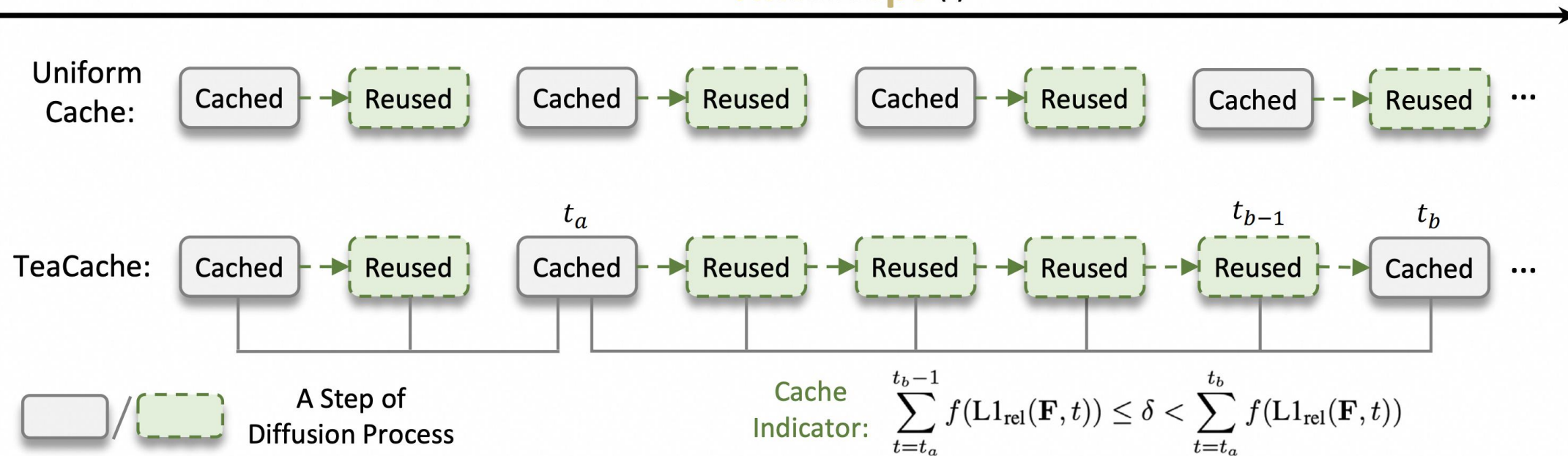
Source: Cache-DiT (Apache-2.0) · PR #5853

TeaCache: Step-level Reuse

TeaCache across steps #5840)

- TeaCache: timestep embeddings predict output change (rel-L1) thresh 0.18: -16.6%, LPIPS 0.120

Timesteps (t)

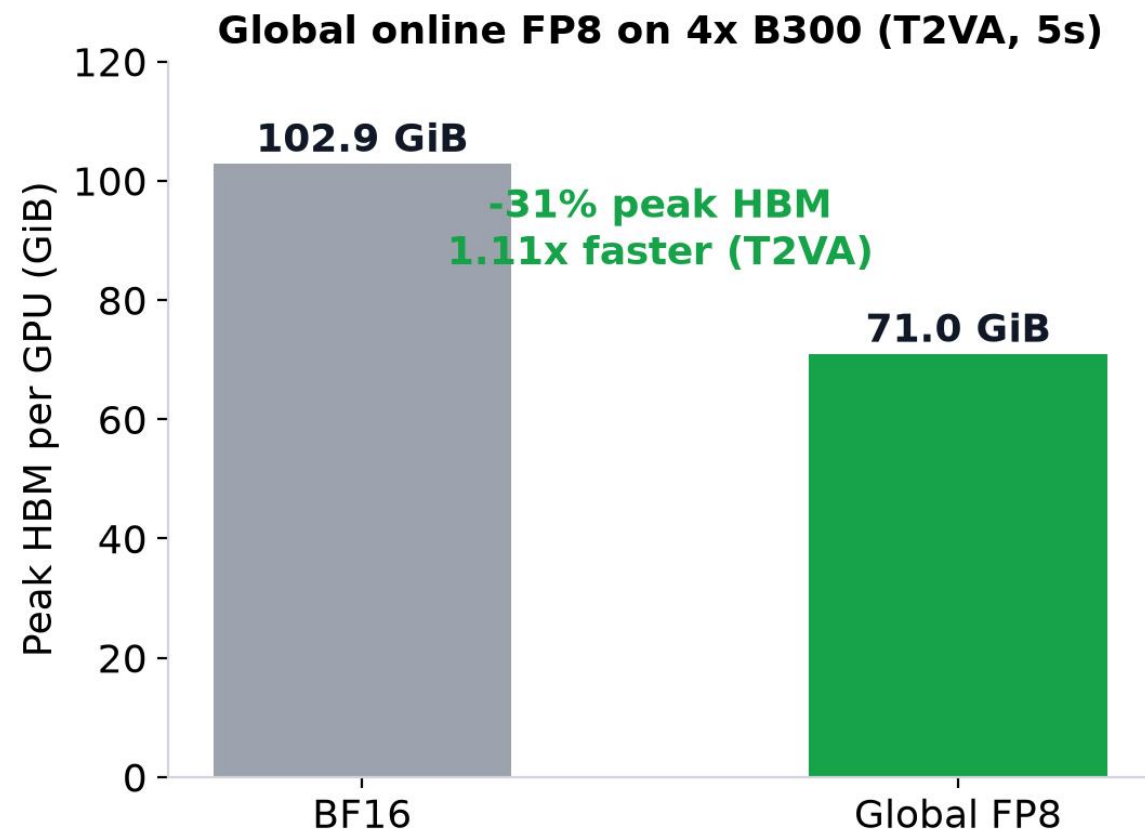
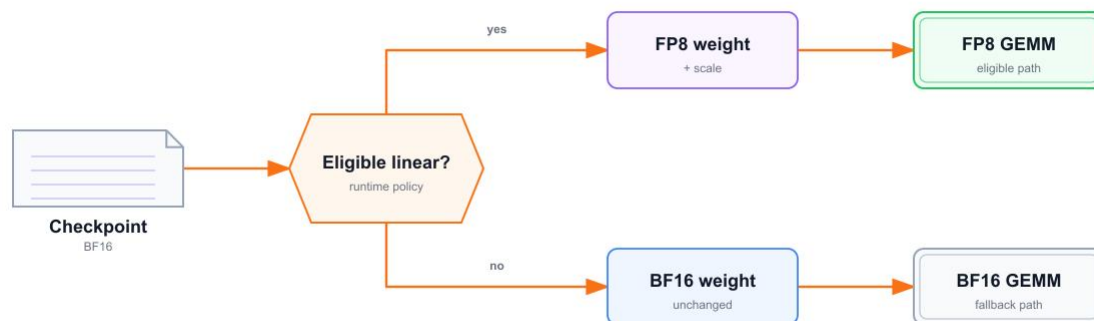


Top: TeaCache (Apache-2.0) · bottom: request-level Cache-DiT on 4×H200 · PR #5840 #5853

Online FP8 Quantization

Checkpoint stays BF16 on disk; eligible linears quantized at runtime

- DiT (#5737) → global: DiT + Qwen3-VL decoder (#5910)
- Norms / RoPE / embeddings / both VAEs keep checkpoint precision
- 4×B300: peak HBM −31%, T2VA 1.11× faster (65.6 → 59.0 s)
- ignored_layers keeps any linear in BF16
- Next: ModelOpt FP8 · encoder NVFP4 · VAE FP8 · W4A4



Source: PR #5737 #5910 · [lishunyang12.github.io/vllm-omni-rankings](https://github.com/lshunyang12/vllm-omni-rankings)

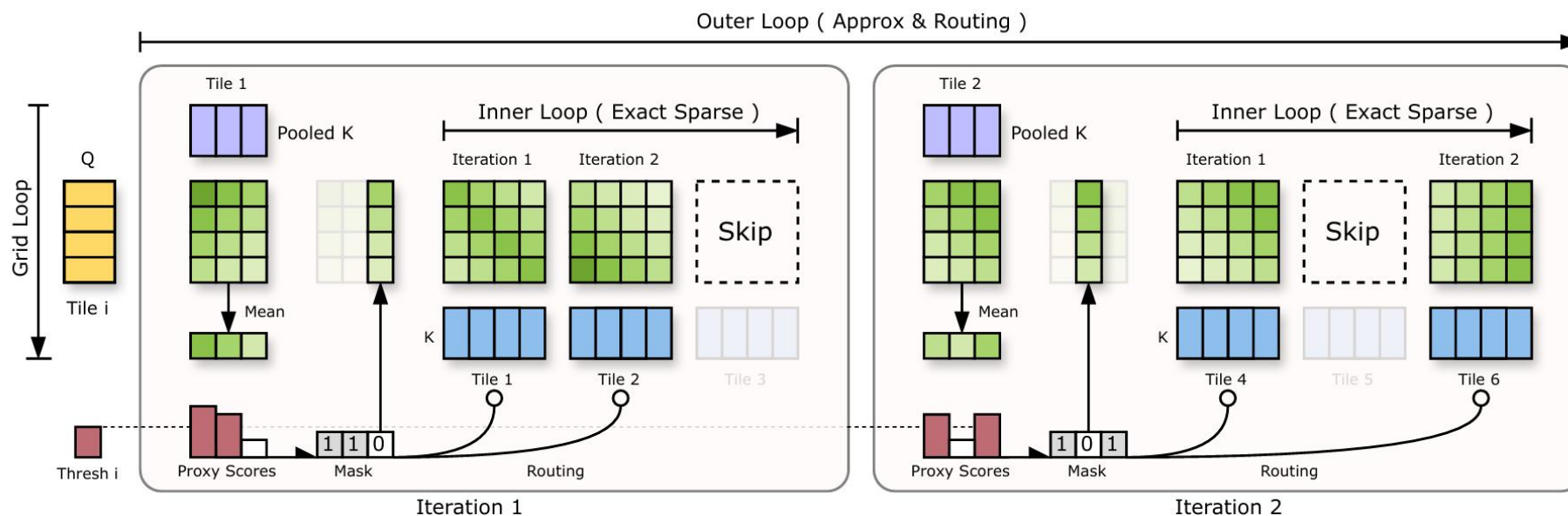
OPTIMIZATION

Sol-attn (WIP#5851)

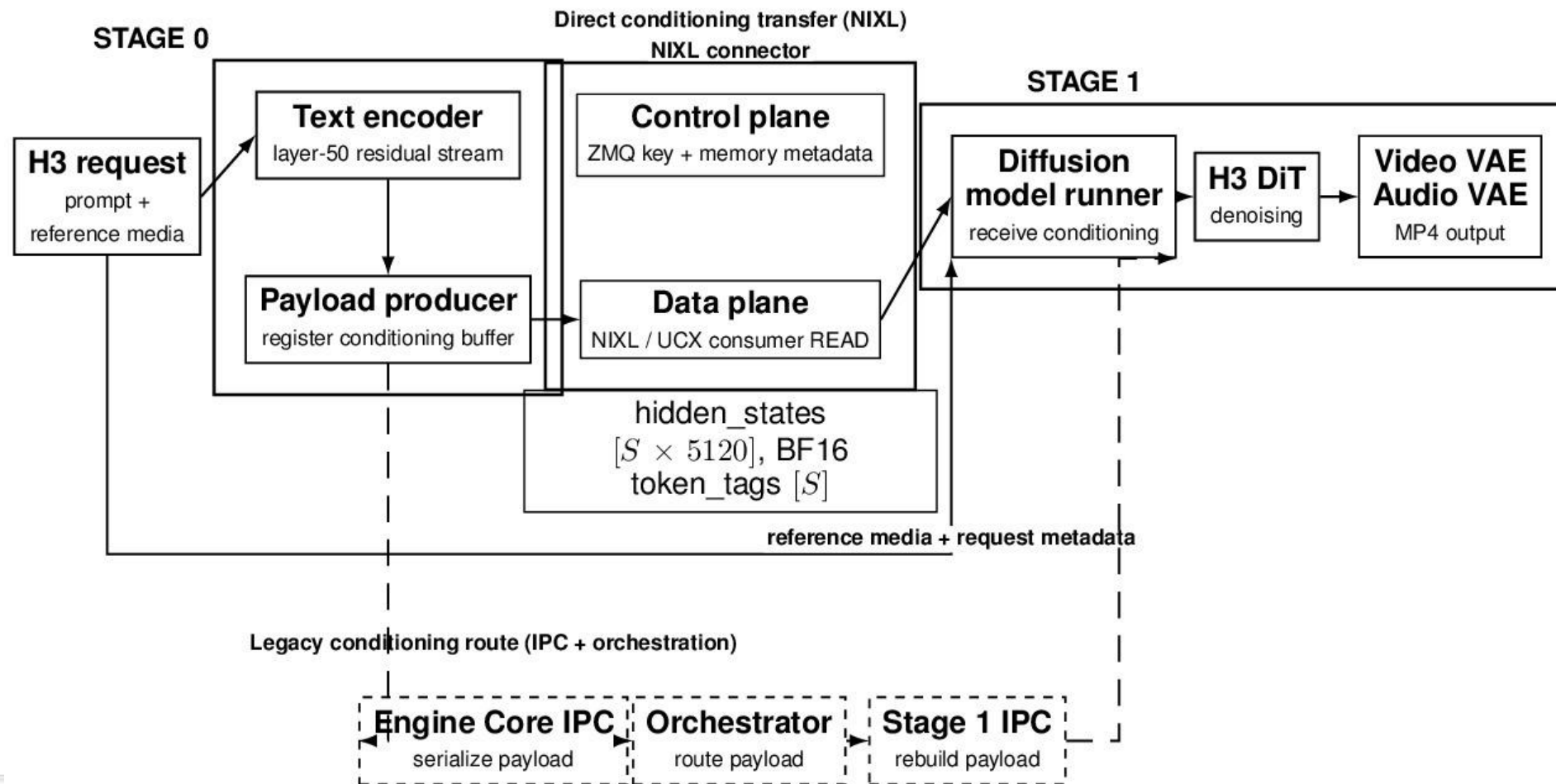
- Sparse attention with 1.1~1.2x speedup
- Generation quality strongly depends on pruning strategy**

Main speed/quality result

Configuration	Median (s)	Speedup	Latency reduction	SSIM	PSNR (dB)	LPIPS	Quality
Dense cuDNN	142.49	1.000×	0.00%	1.00000	inf	0.00000	PASS
Sol-Attn recommended	129.91	1.097×	8.83%	0.93344	33.28	0.08563	PASS
Sol-Attn balanced	124.95	1.140×	12.31%	0.91780	31.49	0.11771	PASS
Sol-Attn fastest passing	122.75	1.161×	13.86%	0.90985	30.58	0.11525	PASS
Sol-Attn aggressive	118.29	1.205×	16.99%	0.87586	27.00	0.22556	FAIL



Text encoder disaggregation (WIP#5885)



OPTIMIZATION

Turbo LoRA support (WIP#6017)

TURBO_LORA=\$(hf download lightx2v/Minimax-h3-Turbo \minimax_h3_fl2v_turbo_4step_v1.0_768p_bf16.safetensors)

```
vllm serve MiniMaxAI/MiniMax-H3 \
--omni \
--trust-remote-code \
--task-type fl2va \
--dynamic-lora "$TURBO_LORA=1.0"
```

- Converged with only 4 steps
- **~7x speedup**

Case	LoRA execution	NFE	Stage 0 p50	Five-run range
Base reference	None	49	69.466 s	69.407–69.590 s
No-LoRA control	None	4	9.178 s	9.097–9.325 s
Turbo	Prefused	4	8.954 s	8.941–8.973 s
Turbo	Dynamic	4	9.505 s	9.481–9.543 s
Turbo + test-only Anime	Prefused Turbo + dynamic Anime	4	9.599 s	9.525–9.622 s
Turbo + test-only Anime	Dynamic Turbo + dynamic Anime	4	9.736 s	9.569–9.832 s



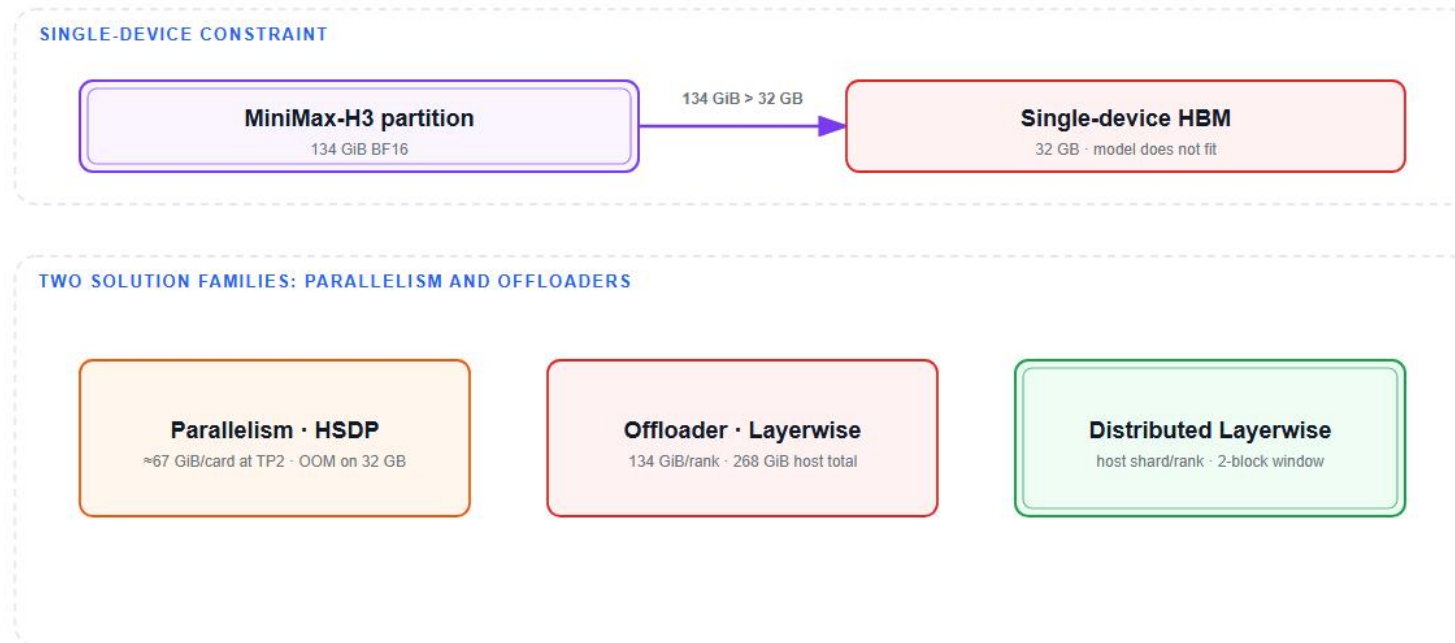
Distributed Layerwise Offload

Serving DiT models larger than single-device HBM

The Problem

Neither parallelism nor naive offload alone fits 100 GB+ DiT models

- Minimax-H3 134 GB BF16 > 32 GB HBM (RTX 5090)
- Naive offload: $N \times$ model in host RAM for per device



Source: vLLM-Omni DLO blog (2026-08-17)

DLO · Host Memory: mmap + Sharding

Load once, share the page cache; pin only 1 /dp_size per rank

- Meta-device init → 2mmap views into one shared OS page cache
- Cold-start host peak: 178 → 47 GB (−73%, Cosmos3-Nano DP4)
- Each rank pins only its 1 /dp_size shard (DMA-ready)
- Total pinned host memory: 4×124 GB → 124 GB

Cold-start Host Memory: Private Copies vs mmap

Cosmos3-Nano · 33 GB weights · DP4



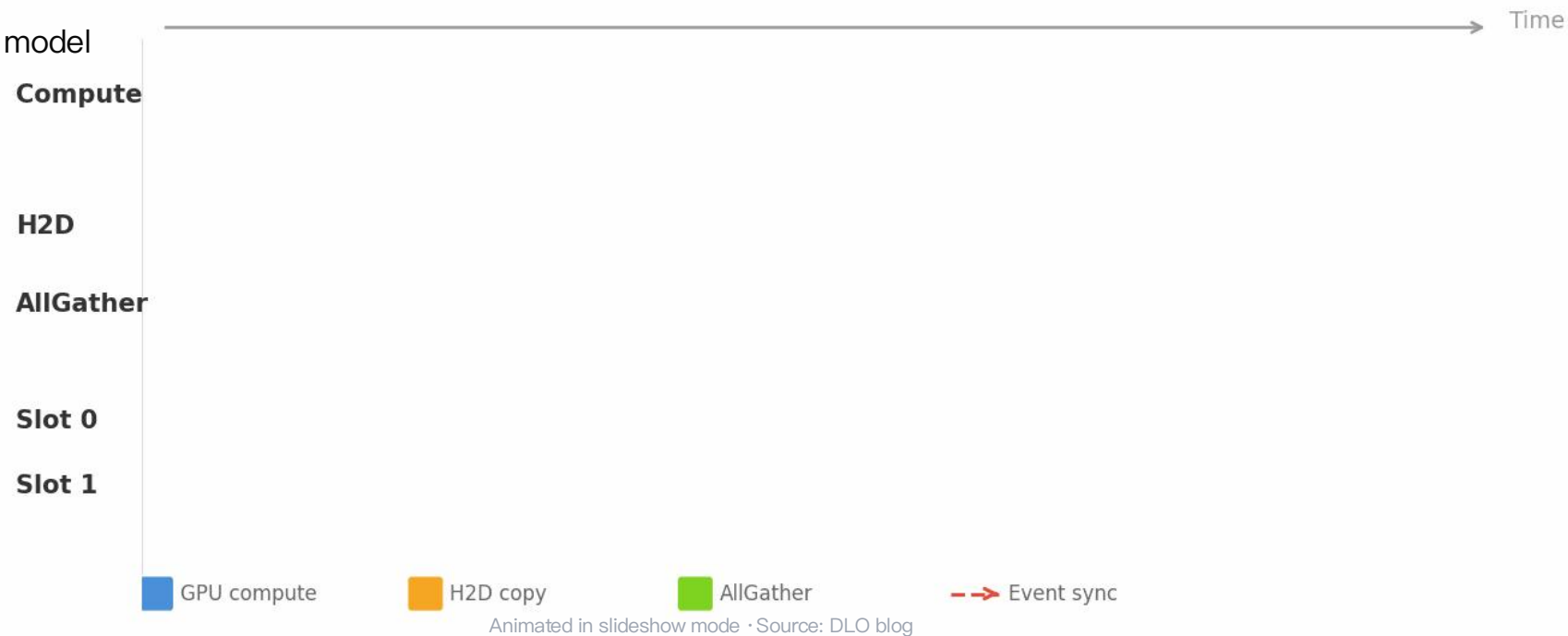
Baseline arithmetic: 132 GB private copies + 33 GB cache + ~13 GB overhead = 178 GB observed peak

Source: DLO blog · distributed_layerwise_backend.py · PR #6213

DLO · Double-Buffer Prefetch

Exactly 2 layers on HBM at any time — independent of layer count

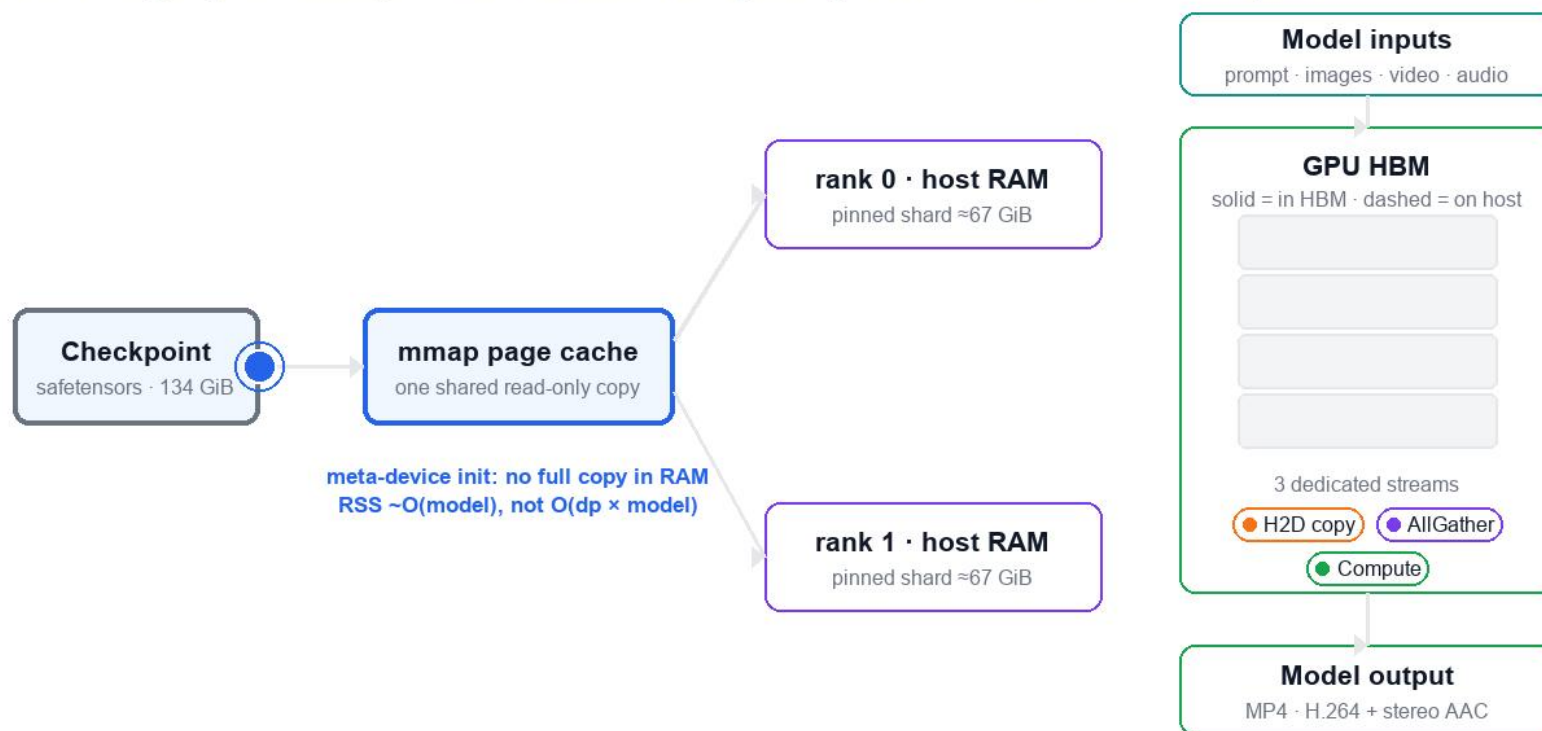
- Three streams overlap: compute / H2D / AllGather
- Buffers sized once to the largest block, reused per layer
- HBM weights: $\approx 2-3$ GB even for the 64B model



DLO · DP Multi-Concurrency

AllGather is request-independent — one broadcast RPC, N requests in parallel · 4 concurrent → 3.3× throughput

DLO weight path: mmap → 1× host shards → per-layer H2D + AllGather → 2-layer window



Source: DLO blog · PR #5864

DLO Results

Model grows 3.8× — peak HBM grows only 22%

- 17B → 64B: peak HBM 23.1 → 28.1 GB
- Byte-identical outputs on Ascend 910B3 and B300
- Extrapolation: 400 GB / 200B-class fits a 2 TB host

DLO on MiniMax-H3: HBM Stays Bounded

Measured · 2× RTX 5090 (32 GB) · 8× B300 · 768×1344, 124f, 50 steps

DISTRIBUTED LAYERWISE OFFLOAD · TWO WEIGHT LAYERS RESIDENT

2× RTX 5090 · rank-local
22.6 GiB peak · 32 GB card · ~10 GiB free

8× B300 · DP1×SP8
26.4 GiB peak · AllGather route

across topologies
peak 22.6 → 26.4 GiB · flat

RESIDENT · NO OFFLOAD

2× 5090 TP2 resident
~67 GiB → OOM

1× B300 resident
90.8 GiB measured peak

Sources: PR #5764 run record · B300 study · recipe 1×B300 capacity check

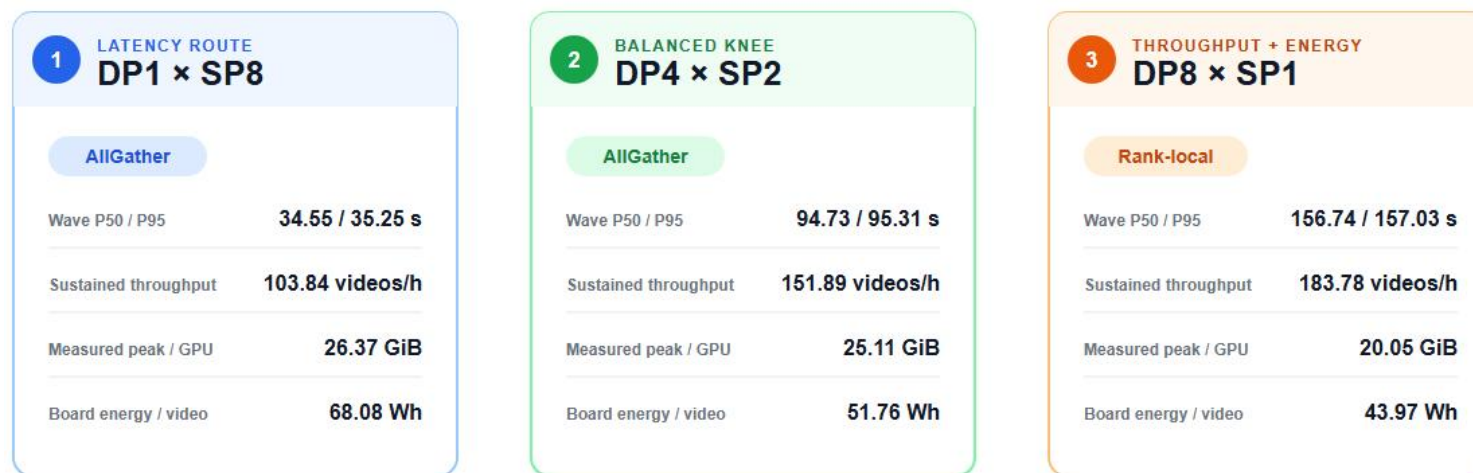
Source: DLO blog · 720p 10s workload

H3 on 8× B300: Topology-Aware DLO

DP1 × SP8 AllGather = lowest latency (P50 34.6 s) · DP8 × SP1 rank-local = 183.8 videos/h, 43.97 Wh/video · AllGather flips sign at DP8

MiniMax-H3: choose DLO mode with the DP × SP topology

8× NVIDIA B300 · T2VA 768×1344 · 124 frames + stereo audio · 50 requested steps · n=20 waves per selected route



AllGather crossover (paired n=5 per mode)

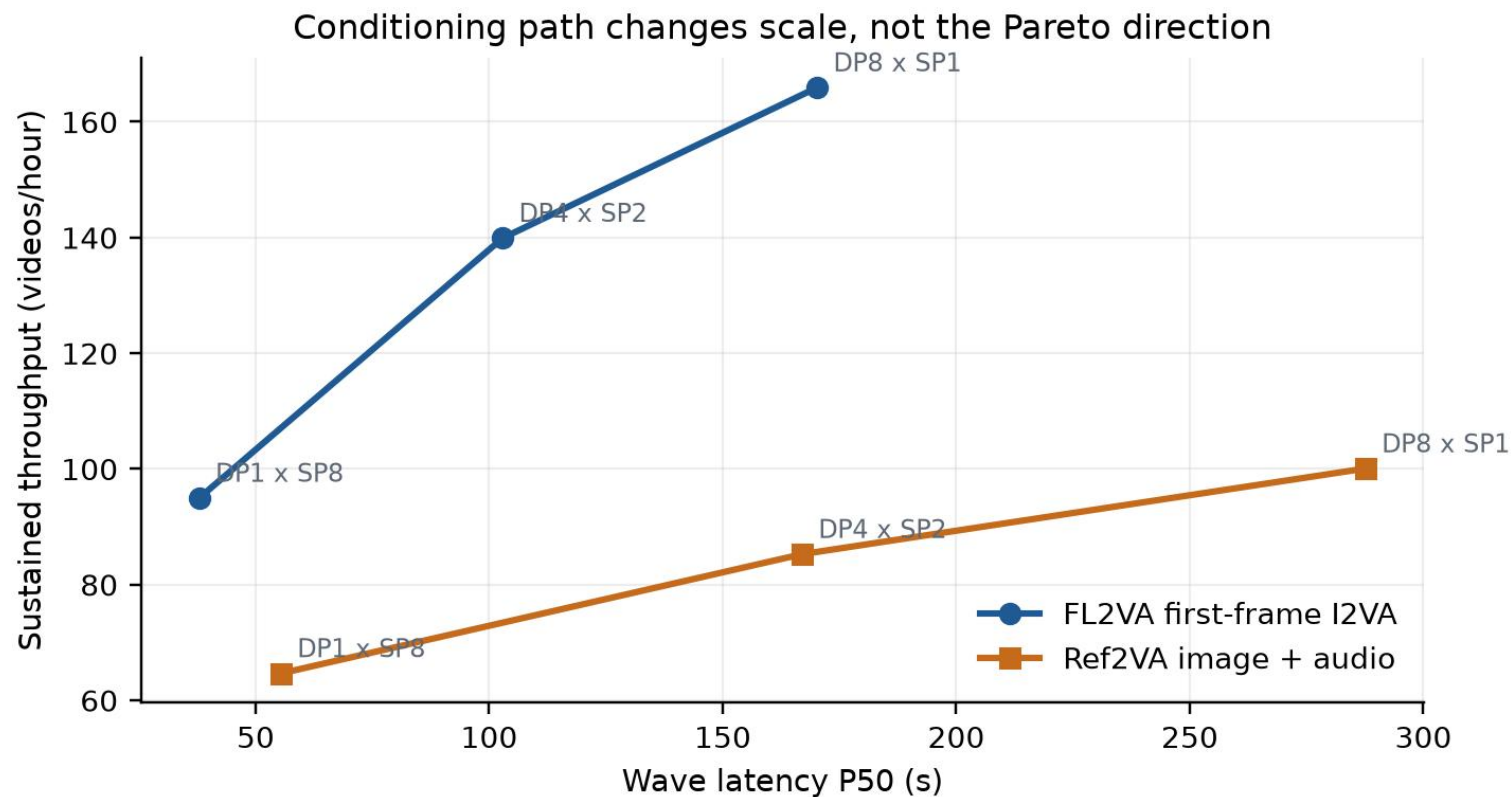
DP1 × SP8 +129.4% throughput · -56.6% P50 DP4 × SP2 +2.2% throughput · -2.2% P50 DP8 × SP1 -4.1% throughput · peak 20.03→94.03 GiB

Negative P50 change is favorable. The DP8 result is why rank-local DLO, not AllGather, is selected for the high-throughput route.

Source: MiniMax-H3 B300 study (lishunyang12/vllm-omni-rankings) · 20 waves/route · 768×1344 × 124f · BF16 · 50 steps

FL2VA & Ref2VA Frontiers

The DP1×SP8 → DP4×SP2 → DP8×SP1 ordering holds across tasks



Source: MiniMax-H3 B300 study · n=5 waves per route

Recipe: Serve & Call on 2× Consumer GPUs

The validated 2× RTX 5090 serving command, then one POST per request (#5764)

1 • serve

```
vllm serve MiniMaxAI/MiniMax-H3 \  
  --omni --trust-remote-code \  
  --num-gpus 2 --tensor-parallel-size 2 \  
  --text-encoder-tp-size 2 \  
  --vae-patch-parallel-size 2 --vae-parallel-mode tile \  
  --vae-use-tiling \  
  --enable-distributed-layerwise-offload \  
  --dlo-resident-layers 20
```

2 • call

```
curl -X POST http://127.0.0.1:8091/v1/videos/sync \  
  -F 'prompt=In a snowy blue-purple forest, Ori \  
    carefully walks past a sleeping giant...' \  
  -F 'width=1344' -F 'height=768' -F 'fps=24' \  
  -F 'num_inference_steps=50' -F 'flow_shift=12' \  
  -F 'seed=1101' \  
  -F 'extra_params={"task":"t2va","duration":8.7, \  
    "audio_flow_shift":3.0}' \  
  -o t2va.mp4
```

Source: [recipes/MiniMaxAI/MiniMax-H3.md](#) • Two 24/32 GB GPUs + HTTP API examples

H3 on Consumer GPUs

134 GiB checkpoint serves on 2× RTX 5090

- TP2 + rank-local DLO + VAE tiling (#5764)
- 20 DiT blocks resident, 30 tail blocks streamed
- 1344×768 × 124f × 50 steps: 8 min 38 s, no OOM
- Peak ≈22.6 GiB/GPU — ≈10 GB headroom per card

• 已完成并验证：

- 仓库：tmp/vllm-omni, 分支 pr-5764, HEAD ae6577ea, 完整 Git 历史已拉取。
- 环境：/root/tmp/.venv
- 版本：vllm 0.26.0、vllm-omni 0.26.1.dev14+gae6577eaa、torch 2.11.0+cu130
- 服务：http://127.0.0.1:8000, 健康检查返回 200
- tmux 会话：minimax-h3

完整测试请求成功，耗时 8m38s，没有 OOM。观测峰值约 22.6GB/卡，每张 5090 仍有约 10GB 余量。

输出：tmp/vllm-omni/results/minimax-h3-t2va.mp4

- 1344x768
- 124 帧, 24fps
- H.264 视频
- AAC 32kHz 双声道音频
- 完整 ffmpeg 解码通过

日志：tmp/vllm-omni/run/minimax-h3.log

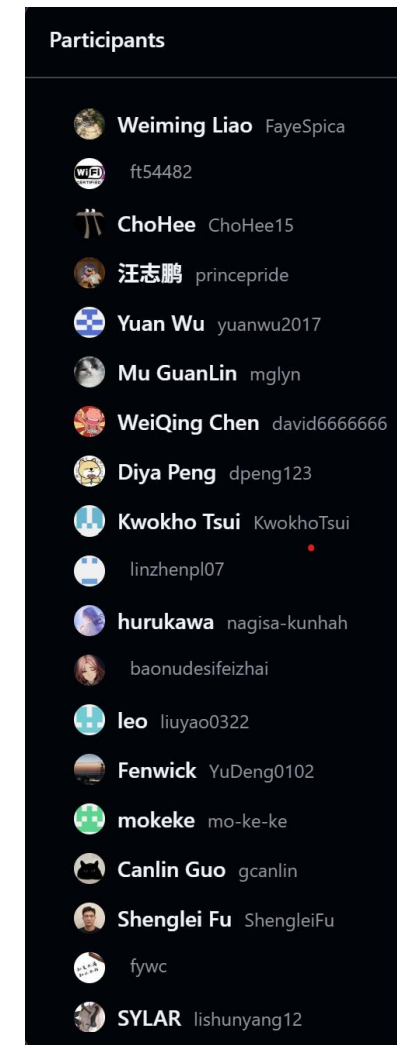
tmux attach -t minimax-h3

tail -f /root/tmp/vllm-omni/run/minimax-h3.log

Source: actual 2× RTX 5090 validation run record (PR #5764)

Thanks for all contributors working on H3!

- Recipe: <https://recipes.vllm.ai/MiniMaxAI/MiniMax-H3>
- Roadmap: <https://github.com/vllm-project/vllm-omni/issues/5700>
- 20+ contributors participated in day-1 optimizations
- 80+ PRs for Minimax-H3 improvements





Networking Hour – vLLM-Omni

Building the **fastest** and **easiest-to-use** open-source
Omni-Modality model inference & serving engine!



<https://github.com/vllm-project/vllm-omni>



<https://slack.vllm.ai>



<https://docs.vllm.ai/projects/vllm-omni>



https://twitter.com/vllm_project



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://discuss.vllm.ai/>



vllm_project



<https://www.zhihu.com/people/vllm-team>



vllm_project



<https://www.linkedin.com/company/vllm-project>



vllm-questions@lists.berkeley.edu



Networking Hour – vLLM-Omni

Building the **fastest** and **easiest-to-use** open-source
Omni-Modality model inference & serving engine!



<https://github.com/vllm-project/vllm-omni>



<https://slack.vllm.ai>



<https://docs.vllm.ai/projects/vllm-omni>



https://twitter.com/vllm_project



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://discuss.vllm.ai/>



vllm_project



<https://www.zhihu.com/people/vllm-team>



vllm_project



<https://www.linkedin.com/company/vllm-project>



vllm-questions@lists.berkeley.edu



Networking Hour – vLLM-Omni

Building the **fastest** and **easiest-to-use** open-source
Omni-Modality model inference & serving engine!



<https://github.com/vllm-project/vllm-omni>



<https://slack.vllm.ai>



<https://docs.vllm.ai/projects/vllm-omni>



https://twitter.com/vllm_project



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://discuss.vllm.ai/>



vllm_project



<https://www.zhihu.com/people/vllm-team>



vllm_project



<https://www.linkedin.com/company/vllm-project>



vllm-questions@lists.berkeley.edu



Networking Hour – vLLM-Omni

Building the **fastest** and **easiest-to-use** open-source
Omni-Modality model inference & serving engine!



<https://github.com/vllm-project/vllm-omni>



<https://slack.vllm.ai>



<https://docs.vllm.ai/projects/vllm-omni>



https://twitter.com/vllm_project



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://discuss.vllm.ai/>



vllm_project



<https://www.zhihu.com/people/vllm-team>



vllm_project



<https://www.linkedin.com/company/vllm-project>



vllm-questions@lists.berkeley.edu



Networking Hour – vLLM-Omni

Building the **fastest** and **easiest-to-use** open-source
Omni-Modality model inference & serving engine!



<https://github.com/vllm-project/vllm-omni>



<https://slack.vllm.ai>



<https://docs.vllm.ai/projects/vllm-omni>



https://twitter.com/vllm_project



<https://blog.vllm.ai>



<https://opencollective.com/vllm>



<https://discuss.vllm.ai/>



vllm_project



<https://www.zhihu.com/people/vllm-team>



vllm_project



<https://www.linkedin.com/company/vllm-project>



vllm-questions@lists.berkeley.edu